

P1028R0: SG14 **status_code** and standard **error** object for P0709 Zero-overhead deterministic exceptions

Document #: P1028R0
Date: 2018-05-06
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposal for the replacement, in new code, of the system header `<system_error>` with a substantially refactored and lighter weight design, which meets modern C++ design and implementation. This paper received the following vote at the May meeting of SG14: 8/2/1/0/0 (SF/WF/N/WA/SA).

A C++ 11 reference implementation of the proposed replacement can be found at <https://github.com/ned14/status-code>. Support for the proposed objects has been wired into Boost.Outcome [1], a library-only implementation of [P0709]. The reference implementation has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64.

Contents

1	Introduction	2
2	Impact on the Standard	3
3	Proposed Design	4
3.1	<code>status_code_domain</code>	4
3.2	<code>status_code<void></code>	7
3.3	<code>status_code<DomainType></code>	8
3.4	<code>status_code<erased<INTEGRAL_TYPE>></code>	10
3.5	Exception types	11
3.6	Generic code	12
3.7	OS specific codes, and erased system code	14
3.8	Proposed <code>std::error</code> object	15
4	Design decisions, guidelines and rationale	17
4.1	Do not <code>#include <string></code>	17
4.2	All <code>constexpr</code> sourcing, construction and destruction	17
4.3	Header only libraries can now safely define custom code categories	18
4.4	No more <code>if(!ec)</code>	19
4.5	No more filtering codes returned by system APIs	19
4.6	All comparisons between codes are now semantic, not literal	20

4.7	<code>std::error_condition</code> is removed entirely	20
4.8	<code>status_code</code> 's value type is set by its domain	20
4.9	<code>status_code<DomainType></code> is type erasable	20
4.10	More than one 'system' error coding domain: <code>system_code</code>	21
4.11	<code>std::errc</code> is now represented as type alias <code>generic_code</code>	21
5	Technical specifications	21
6	Frequently asked questions	22
6.1	Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?	22
7	Acknowledgements	22
8	References	22

1 Introduction

The `<system_error>` header entered the C++ standard in the C++ 11 standard, the idea for which having been split off from the Filesystem TS proposal into its own [N2066] proposal back in 2006. Despite its relative lack of direct usage by the C++ userbase, according to [2], `<system_error>` has become one of the most common internal dependencies for all other standard header files, frequently constituting up to 20% of all the tokens brought into the compiler by other standard header files e.g. `<array>`, `<complex>` or `<optional>`. In this sense, it is amongst the most popular system headers in the C++ standard library.

So why would anyone want to replace it? It unfortunately suffers from a number of design problems only now apparent after twelve years of hindsight, which makes it low hanging fruit in the achievement of the 'reduce compile time' and 'alternatives to complicated and/or error-prone features' goals listed in [P0939] *Direction for ISO C++*. We, from Study Group 14 (the GameDev & low latency WG21 working group), listed many of these problems in [P0824], and after an extensive period of consultation with other stakeholders including the Boost C++ Libraries, we thence designed and implemented an improved substitute which does not have those problems. It is this improved design that we propose now.

This proposed library may be useful as the standardised implementation of the lightweight throwable **error** object as proposed by [P0709] *Zero-overhead deterministic exceptions: Throwing values*. It is [P0829] *Freestanding C++* compatible i.e. without dependency on any STL or language facility not usable on embedded systems.

An example of use:

```

1 std::system_code sc; // default constructs to empty
2 native_handle_type h = open_file(path, sc);
3 // Is the code a failure?
4 if(sc.failure())

```

```

5 {
6     // Do semantic comparison to test if this was a file not found failure
7     // This will match any system-specific error codes meaning a file not found
8     if(sc != std::errc::no_such_file_or_directory)
9     {
10         std::cerr << "FATAL: " << sc.message().c_str() << std::endl;
11         std::terminate();
12     }
13 }

```

The above is 100% portable code. Meanwhile, the implementation of `open_file()` might be these:

<pre> 1 // POSIX implementation 2 using native_handle_type = int; 3 native_handle_type open_file(const char *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = ::open(path, O_RDONLY); 8 if(-1 == h) 9 { 10 // posix_code type erases into system_code 11 sc = std::posix_code(errno); 12 } 13 return h; 14 } </pre>	<pre> 1 // Microsoft Windows implementation 2 using native_handle_type = HANDLE; 3 native_handle_type open_file(const wchar_t *path, 4 std::system_code &sc) noexcept 5 { 6 sc.clear(); // clears to empty 7 native_handle_type h = CreateFile(path, 8 GENERIC_READ, 9 FILE_SHARE_READ FILE_SHARE_WRITE 10 FILE_SHARE_DELETE, 11 nullptr, 12 OPEN_EXISTING, 13 FILE_ATTRIBUTE_NORMAL, 14 nullptr 15); 16 if(INVALID_HANDLE_VALUE == h) 17 { 18 // win32_code type erases into system_code 19 sc = std::win32_code(GetLastError()); 20 } 21 return h; 22 } </pre>
---	--

2 Impact on the Standard

The proposed library is a pure-library solution.

There is an optional dependency on a core language enhancement:

1. P1029 *SG14* `[[move_relocates]]` <https://wg21.link/P1029>.

This proposes a new C++ attribute `[[move_relocates]]` which lets the compiler optimise such attributed moves as aggressively as trivially copyable types. If approved, this would enable a large increase in the variety of types directly transportable in the proposed `error` object, specifically the ability to transport `std::exception_ptr` instances directly, a highly desirable feature for improving efficiency of legacy C++ exceptions support under P0709.

3 Proposed Design

3.1 status_code_domain

```
1  /*! The main workhorse of the system_error2 library, can be typed
2  ('status_code<DomainType>'), erased-immutable ('status_code<void>') or
3  erased-mutable ('status_code<erased<T>>').
4
5  Be careful of placing these into containers! Equality and inequality operators are
6  *semantic* not exact. Therefore two distinct items will test true! To help prevent
7  surprise on this, 'operator<' and 'std::hash<>' are NOT implemented in order to
8  trap potential incorrectness. Define your own custom comparison functions for your
9  container which perform exact comparisons.
10 */
11 template <class DomainType> class status_code;
12
13 template <class T>
14 struct is_status_code
15 {
16     static constexpr bool const value;
17 };
18
19 class _generic_code_domain;
20
21 /*! The generic code is a status code with the generic code domain, which is that of 'errc' (POSIX).
22 using generic_code = status_code<_generic_code_domain>;
```

```
1  class status_code_domain
2  {
3      template <class DomainType> friend class status_code;
4
5  public:
6      /*! Type of the unique id for this domain.
7      using unique_id_type = unsigned long long;
8
9      /*! (Potentially thread safe) Reference to a message string.
10     Be aware that you cannot add payload to implementations of this class.
11     You get exactly the 'void *[3]' array to keep state, this is usually
12     sufficient for a 'std::shared_ptr<>' or a 'std::string'.
13     You can install a handler to be called when this object is copied,
14     moved and destructed. This takes the form of a C function pointer.
15     */
16     class string_ref
17     {
18     public:
19         /*! The value type
20         using value_type = const char;
21         /*! The size type
22         using size_type = size_t;
23         /*! The pointer type
24         using pointer = const char *;
25         /*! The const pointer type
26         using const_pointer = const char *;
27         /*! The iterator type
28         using iterator = const char *;
```

```

29     ///! The const iterator type
30     using const_iterator = const char *;
31
32 protected:
33     ///! The operation occurring
34     enum class _thunk_op
35     {
36         copy,
37         move,
38         destruct
39     };
40     ///! The prototype of the handler function. Copies can throw, moves and destructs cannot.
41     using _thunk_spec = void (*)(string_ref *dest, const string_ref *src, _thunk_op op);
42     ///! Pointers to beginning and end of character range
43     pointer _begin{}, _end{};
44     ///! Three 'void*' of state
45     void *_state[3]{}; // at least the size of a shared_ptr
46     ///! Handler for when operations occur
47     const _thunk_spec _thunk{nullptr};
48
49     constexpr explicit string_ref(_thunk_spec thunk) noexcept;
50
51 public:
52     ///! Construct from a C string literal
53     constexpr explicit string_ref(const char *str, size_type len = static_cast<size_type>(-1),
54                                   void *state0 = nullptr, void *state1 = nullptr,
55                                   void *state2 = nullptr, _thunk_spec thunk = nullptr) noexcept;
56     ///! Copy construct the derived implementation.
57     string_ref(const string_ref &o);
58     ///! Move construct the derived implementation.
59     string_ref(string_ref &&o) noexcept;
60     ///! Copy assignment
61     string_ref &operator=(const string_ref &o);
62     ///! Move assignment
63     string_ref &operator=(string_ref &&o) noexcept;
64     ///! Destruction
65     ~string_ref();
66
67     ///! Returns whether the reference is empty or not
68     bool empty() const noexcept;
69     ///! Returns the size of the string
70     size_type size() const noexcept;
71     ///! Returns a null terminated C string
72     value_type *c_str() const noexcept;
73     ///! Returns the beginning of the string
74     iterator begin() noexcept;
75     ///! Returns the beginning of the string
76     const_iterator begin() const noexcept;
77     ///! Returns the beginning of the string
78     const_iterator cbegin() const noexcept;
79     ///! Returns the end of the string
80     iterator end() noexcept;
81     ///! Returns the end of the string
82     const_iterator end() const noexcept;
83     ///! Returns the end of the string
84     const_iterator cend() const noexcept;

```

```

85 };
86
87 /*! A reference counted, threadsafe reference to a message string.
88 */
89 class atomic_refcounted_string_ref : public string_ref
90 {
91     struct _allocated_msg
92     {
93         mutable std::atomic<unsigned> count;
94     };
95     _allocated_msg * &_msg() noexcept;
96     const _allocated_msg * _msg() const noexcept;
97
98     static void _refcounted_string_thunk(string_ref *_dest, const string_ref *_src, _thunk_op op)
99         noexcept;
100
101 public:
102     /*! Construct from a C string literal
103     explicit atomic_refcounted_string_ref(const char *str, size_type len = static_cast<size_type>(-1),
104                                         void *state1 = nullptr, void *state2 = nullptr) noexcept;
105
106 private:
107     unique_id_type _id;
108
109 protected:
110     /*! Use [https://www.random.org/cgi-bin/randbyte?nbytes=8&format=h](https://www.random.org/cgi-bin/
111         randbyte?nbytes=8&format=h) to get a random 64 bit id.
112     Do NOT make up your own value. Do NOT use zero.
113     */
114     constexpr explicit status_code_domain(unique_id_type id) noexcept;
115     /*! No public copying at type erased level
116     status_code_domain(const status_code_domain &) = default;
117     /*! No public moving at type erased level
118     status_code_domain(status_code_domain &&) = default;
119     /*! No public assignment at type erased level
120     status_code_domain &operator=(const status_code_domain &) = default;
121     /*! No public assignment at type erased level
122     status_code_domain &operator=(status_code_domain &&) = default;
123     /*! No public destruction at type erased level
124     ~status_code_domain() = default;
125
126 public:
127     /*! True if the unique ids match.
128     constexpr bool operator==(const status_code_domain &o) const noexcept;
129     /*! True if the unique ids do not match.
130     constexpr bool operator!=(const status_code_domain &o) const noexcept;
131     /*! True if this unique is lower than the other's unique id.
132     constexpr bool operator<(const status_code_domain &o) const noexcept;
133
134     /*! Returns the unique id used to identify identical category instances.
135     constexpr unique_id_type id() const noexcept;
136     /*! Name of this category.
137     virtual string_ref name() const noexcept = 0;
138
139 protected:

```

```

139  ///! True if code means failure.
140  virtual bool _failure(const status_code<void> &code) const noexcept = 0;
141  ///! True if code is (potentially non-transitively) equivalent to another code in another domain.
142  virtual bool _equivalent(const status_code<void> &code1, const status_code<void> &code2) const
        noexcept = 0;
143  ///! Returns the generic code closest to this code, if any.
144  virtual generic_code _generic_code(const status_code<void> &code) const noexcept = 0;
145  ///! Return a reference to a string textually representing a code.
146  virtual string_ref _message(const status_code<void> &code) const noexcept = 0;
147  ///! Throw a code as a C++ exception.
148  virtual void _throw_exception(const status_code<void> &code) const = 0;
149  };

```

3.2 status_code<void>

```

1  /*! A type erased lightweight status code reflecting empty, success, or failure.
2  Differs from 'status_code<erased<>>' by being always available irrespective of
3  the domain's value type, but cannot be copied, moved, nor destructed. Thus one
4  always passes this around by const lvalue reference.
5  */
6  template <> class status_code<void>
7  {
8      template <class T> friend class status_code;
9
10 public:
11     ///! The type of the domain.
12     using domain_type = status_code_domain;
13     ///! The type of the status code.
14     using value_type = void;
15     ///! The type of a reference to a message string.
16     using string_ref = typename domain_type::string_ref;
17
18 protected:
19     const status_code_domain *_domain{nullptr};
20
21 protected:
22     ///! No default construction at type erased level
23     status_code() = default;
24     ///! No public copying at type erased level
25     status_code(const status_code &) = default;
26     ///! No public moving at type erased level
27     status_code(status_code &&) = default;
28     ///! No public assignment at type erased level
29     status_code &operator=(const status_code &) = default;
30     ///! No public assignment at type erased level
31     status_code &operator=(status_code &&) = default;
32     ///! No public destruction at type erased level
33     ~status_code() = default;
34
35     ///! Used to construct a non-empty type erased status code
36     constexpr explicit status_code(const status_code_domain *v) noexcept;
37
38 public:
39     ///! Return the status code domain.

```

```

40 constexpr const status_code_domain &domain() const noexcept;
41 /// True if the status code is empty.
42 constexpr bool empty() const noexcept;
43
44 /// Return a reference to a string textually representing a code.
45 string_ref message() const noexcept;
46 /// True if code means success.
47 bool success() const noexcept;
48 /// True if code means failure.
49 bool failure() const noexcept;
50 /*! True if code is strictly (and potentially non-transitively) semantically equivalent to
51 another code in another domain.
52
53 Note that usually non-semantic i.e. pure value comparison is used when the other
54 status code has the same domain. As 'equivalent()' will try mapping to generic code,
55 this usually captures when two codes have the same semantic meaning in 'equivalent()'.
56 */
57 template <class T> bool strictly_equivalent(const status_code<T> &o) const noexcept;
58 /*! True if code is equivalent, by any means, to another code in another domain
59 (guaranteed transitive).
60
61 Firstly 'strictly_equivalent()' is run in both directions. If neither succeeds, each domain
62 is asked for the equivalent generic code and those are compared.
63 */
64 template <class T> inline bool equivalent(const status_code<T> &o) const noexcept;
65 /// Throw a code as a C++ exception.
66 void throw_exception() const;
67 };

```

3.3 status_code<DomainType>

```

1  /*! A lightweight, typed, status code reflecting empty, success, or failure.
2  This is the main workhorse of the system_error2 library.
3
4  An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one
5  of the constructors. If it is found, and it generates a status code compatible with this
6  status code, implicit construction is made available.
7  */
8  template <class DomainType>
9  requires(
10     (!std::is_default_constructible<typename DomainType::value_type>::value
11      || std::is_nothrow_default_constructible<typename DomainType::value_type>::value)
12     && (!std::is_move_constructible<typename DomainType::value_type>::value
13      || std::is_nothrow_move_constructible<typename DomainType::value_type>::value)
14     && std::is_nothrow_destructible<typename DomainType::value_type>::value
15  )
16  class status_code : public status_code<void>
17  {
18     template <class T> friend class status_code;
19
20  public:
21     /// The type of the domain.
22     using domain_type = DomainType;
23     /// The type of the status code.

```



```

24     using value_type = typename domain_type::value_type;
25     ///! The type of a reference to a message string.
26     using string_ref = typename domain_type::string_ref;
27
28 protected:
29     value_type _value{};
30
31 public:
32     ///! Default construction to empty
33     status_code() = default;
34     ///! Copy constructor
35     status_code(const status_code &) = default;
36     ///! Move constructor
37     status_code(status_code &&) = default;
38     ///! Copy assignment
39     status_code &operator=(const status_code &) = default;
40     ///! Move assignment
41     status_code &operator=(status_code &&) = default;
42     ~status_code() = default;
43
44     ///! Implicit construction from any type where an ADL discovered
45     ///! 'make_status_code(T, Args ...)' returns a 'status_code'.
46     template <class T, class... Args>
47     constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>(),
48         std::declval<Args>()...)));
49
50     ///! Explicit in-place construction.
51     template <class... Args>
52     constexpr explicit status_code(in_place_t /*unused */, Args &&... args) noexcept(std::
53         is_nothrow_constructible<value_type, Args &&...>::value);
54     ///! Explicit in-place construction from initializer list.
55     template <class T, class... Args>
56     constexpr explicit status_code(in_place_t /*unused */, std::initializer_list<T> il, Args &&... args)
57         noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args &&...>::
58             value);
59     ///! Explicit copy construction from a 'value_type'.
60     constexpr explicit status_code(const value_type &v) noexcept(std::is_nothrow_copy_constructible<
61         value_type>::value);
62     ///! Explicit move construction from a 'value_type'.
63     constexpr explicit status_code(value_type &&v) noexcept(std::is_nothrow_move_constructible<
64         value_type>::value);
65     /*! Explicit construction from an erased status code. Available only if
66     'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased><>>)'.
67     Does not check if domains are equal.
68     */
69     template <class ErasedType>
70     constexpr explicit status_code(const status_code<erased<ErasedType>> &v) noexcept(std::
71         is_nothrow_copy_constructible<value_type>::value);
72
73     ///! Assignment from a 'value_type'.
74     constexpr status_code &operator=(const value_type &v) noexcept(std::is_nothrow_copy_assignable<
75         value_type>::value);
76
77     // Replace the type erased implementations with type aware implementations for better codegen
78     ///! Return the status code domain.
79     constexpr const domain_type &domain() const noexcept;
80     ///! Return a reference to a string textually representing a code.

```

```

72 string_ref message() const noexcept;
73
74 ///! Reset the code to empty.
75 constexpr void clear() noexcept;
76
77 ///! Return a reference to the 'value_type'.
78 constexpr value_type &value() & noexcept;
79 ///! Return a reference to the 'value_type'.
80 constexpr value_type &&value() && noexcept;
81 ///! Return a reference to the 'value_type'.
82 constexpr const value_type &value() const &noexcept;
83 ///! Return a reference to the 'value_type'.
84 constexpr const value_type &&value() const &&noexcept;
85 };

```

3.4 status_code<erased<INTEGRAL_TYPE>>

```

1  template <class ErasedType>
2  requires(std::is_integral<ErasedType>::value)
3  struct erased
4  {
5      using value_type = ErasedType;
6  };
7
8  /*! Type erased status_code, but copyable/movable/destructible unlike 'status_code<void>'.
9  Available only if 'erased<>' is available, which is when the domain's type is trivially
10 copyable, and if the size of the domain's typed error code is less than or equal to
11 this erased error code.
12
13 An ADL discovered helper function 'make_status_code(T, Args...)' is looked up by one of the
14 constructors. If it is found, and it generates a status code compatible with this status code,
15 implicit construction is made available.
16 */
17 template <class ErasedType> class status_code<erased<ErasedType>> : public status_code<void>
18 {
19     template <class T> friend class status_code;
20
21 public:
22     ///! The type of the domain (void, as it is erased).
23     using domain_type = void;
24     ///! The type of the erased status code.
25     using value_type = ErasedType;
26     ///! The type of a reference to a message string.
27     using string_ref = typename _status_code<void>::string_ref;
28
29 protected:
30     value_type _value{};
31
32 public:
33     ///! Default construction to empty
34     status_code() = default;
35     ///! Copy constructor
36     status_code(const status_code &) = default;
37     ///! Move constructor

```

```

38     status_code(status_code &&) = default;
39     /// Copy assignment
40     status_code &operator=(const status_code &) = default;
41     /// Move assignment
42     status_code &operator=(status_code &&) = default;
43     ~status_code() = default;
44
45     /// Implicit copy construction from any other status code if its value type is trivially copyable
46     /// and it would fit into our storage
47     template <class DomainType>
48     constexpr status_code(const status_code<DomainType> &v) noexcept;
49     /// Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
50     /// returns a 'status_code'.
51     template <class T, class... Args>
52     constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::declval<T>(),
53                                     std::declval<Args>()...)));
54     /// Reset the code to empty.
55     constexpr void clear() noexcept;
56     /// Return the erased 'value_type' by value.
57     constexpr value_type value() const noexcept;
58 };
59
60 /// True if the status code's are semantically equal via 'equivalent()'.
61 template <class DomainType1, class DomainType2> inline bool operator==(const status_code<DomainType1>
62     &a, const status_code<DomainType2> &b) noexcept;
63 /// True if the status code's are not semantically equal via 'equivalent()'.
64 template <class DomainType1, class DomainType2> inline bool operator!=(const status_code<DomainType1>
65     &a, const status_code<DomainType2> &b) noexcept;
66 /// True if the status code's are semantically equal via 'equivalent()' to the generic code.
67 template <class DomainType1> inline bool operator==(const status_code<DomainType1> &a, errc b)
68     noexcept;
69 /// True if the status code's are semantically equal via 'equivalent()' to the generic code.
70 template <class DomainType1> inline bool operator==(errc a, const status_code<DomainType1> &b)
71     noexcept;
72 /// True if the status code's are not semantically equal via 'equivalent()' to the generic code.
73 template <class DomainType1> inline bool operator!=(const status_code<DomainType1> &a, errc b)
74     noexcept;
75 /// True if the status code's are not semantically equal via 'equivalent()' to the generic code.
76 template <class DomainType1> inline bool operator!=(errc a, const status_code<DomainType1> &b)
77     noexcept;

```

3.5 Exception types

```

1  /*! Exception type representing a thrown status_code
2  */
3  template <class DomainType> class status_error;
4
5  /*! The erased type edition of status_error.
6  */
7  template <> class status_error<void> : public std::exception
8  {
9  protected:
10     /// Constructs an instance. Not publicly available.
11     status_error() = default;

```

```

12  ///! Copy constructor. Not publicly available
13  status_error(const status_error &) = default;
14  ///! Move constructor. Not publicly available
15  status_error(status_error &&) = default;
16  ///! Copy assignment. Not publicly available
17  status_error &operator=(const status_error &) = default;
18  ///! Move assignment. Not publicly available
19  status_error &operator=(status_error &&) = default;
20  ///! Destructor. Not publicly available.
21  ~status_error() = default;
22
23  public:
24      ///! The type of the status domain
25      using domain_type = void;
26      ///! The type of the status code
27      using status_code_type = status_code<void>;
28  };
29
30  /*! Exception type representing a thrown status_code
31  */
32  template <class DomainType> class status_error : public status_error<void>
33  {
34      status_code<DomainType> _code;
35      typename DomainType::string_ref _msgref;
36
37  public:
38      ///! The type of the status domain
39      using domain_type = DomainType;
40      ///! The type of the status code
41      using status_code_type = status_code<DomainType>;
42
43      ///! Constructs an instance
44      explicit status_error(status_code<DomainType> code);
45
46      ///! Return an explanatory string
47      virtual const char *what() const noexcept override;
48
49      ///! Returns a reference to the code
50      const status_code_type &code() const &;
51      ///! Returns a reference to the code
52      status_code_type &code() &;
53      ///! Returns a reference to the code
54      const status_code_type &&code() const &&;
55      ///! Returns a reference to the code
56      status_code_type &&code() &&;
57  };

```

3.6 Generic code

```

1  ///! The generic error coding (POSIX)
2  enum class errc : int
3  {
4      success = 0,
5      unknown = -1,

```

```

6
7 address_family_not_supported = EAFNOSUPPORT,
8 address_in_use = EADDRINUSE,
9 address_not_available = EADDRNOTAVAIL,
10 already_connected = EISCONN,
11 argument_list_too_long = E2BIG,
12 argument_out_of_domain = EDOM,
13 bad_address = EFAULT,
14 bad_file_descriptor = EBADF,
15 bad_message = EBADMSG,
16 broken_pipe = EPIPE,
17 connection_aborted = ECONNABORTED,
18 connection_already_in_progress = EALREADY,
19 connection_refused = ECONNREFUSED,
20 connection_reset = ECONNRESET,
21 cross_device_link = EXDEV,
22 destination_address_required = EDESTADDRREQ,
23 device_or_resource_busy = EBUSY,
24 directory_not_empty = ENOTEMPTY,
25 executable_format_error = ENOEXEC,
26 file_exists = EEXIST,
27 file_too_large = EFBIG,
28 filename_too_long = ENAMETOOLONG,
29 function_not_supported = ENOSYS,
30 host_unreachable = EHOSTUNREACH,
31 identifier_removed = EIDRM,
32 illegal_byte_sequence = EILSEQ,
33 inappropriate_io_control_operation = ENOTTY,
34 interrupted = EINTR,
35 invalid_argument = EINVAL,
36 invalid_seek = ESPIPE,
37 io_error = EIO,
38 is_a_directory = EISDIR,
39 message_size = EMSGSIZE,
40 network_down = ENETDOWN,
41 network_reset = ENETRESET,
42 network_unreachable = ENETUNREACH,
43 no_buffer_space = ENOBUFS,
44 no_child_process = ECHILD,
45 no_link = ENOLINK,
46 no_lock_available = ENOLCK,
47 no_message = ENOMSG,
48 no_protocol_option = ENOPROTOOPT,
49 no_space_on_device = ENOSPC,
50 no_stream_resources = ENOSR,
51 no_such_device_or_address = ENXIO,
52 no_such_device = ENODEV,
53 no_such_file_or_directory = ENOENT,
54 no_such_process = ESRCH,
55 not_a_directory = ENOTDIR,
56 not_a_socket = ENOTSOCK,
57 not_a_stream = ENOSTR,
58 not_connected = ENOTCONN,
59 not_enough_memory = ENOMEM,
60 not_supported = ENOTSUP,
61 operation_cancelled = ECANCELED,

```

```

62 operation_in_progress = EINPROGRESS,
63 operation_not_permitted = EPERM,
64 operation_not_supported = EOPNOTSUPP,
65 operation_would_block = EWOULDBLOCK,
66 owner_dead = EOWNERDEAD,
67 permission_denied = EACCES,
68 protocol_error = EPROTO,
69 protocol_not_supported = EPROTONOSUPPORT,
70 read_only_file_system = EROFS,
71 resource_deadlock_would_occur = EDEADLK,
72 resource_unavailable_try_again = EAGAIN,
73 result_out_of_range = ERANGE,
74 state_not_recoverable = ENOTRECOVERABLE,
75 stream_timeout = ETIME,
76 text_file_busy = ETXTBSY,
77 timed_out = ETIMEDOUT,
78 too_many_files_open_in_system = ENFILE,
79 too_many_files_open = EMFILE,
80 too_many_links = EMLINK,
81 too_many_symbolic_link_levels = ELOOP,
82 value_too_large = EOVERFLOW,
83 wrong_protocol_type = EPROTOTYPE
84 };
85
86 ///! A specialisation of 'status_error' for the generic code domain.
87 using generic_error = status_error<generic_code_domain>;
88 ///! A constexpr source variable for the generic code domain, which is that of 'errc'
89 ///! (POSIX). Returned by '_generic_code_domain::get()'.
90 constexpr _generic_code_domain generic_code_domain;
91 // Enable implicit construction of generic_code from errc
92 constexpr inline generic_code make_status_code(errc c) noexcept;

```

3.7 OS specific codes, and erased system code

```

1 ///! A POSIX error code, those returned by 'errno'.
2 using posix_code = status_code<posix_code_domain>;
3 ///! A specialisation of 'status_error' for the POSIX error code domain.
4 using posix_error = status_error<posix_code_domain>;
5
6 #ifdef _WIN32
7 ///! (Windows only) A Win32 error code, those returned by 'GetLastError()'.
8 using win32_code = status_code<win32_code_domain>;
9 ///! (Windows only) A specialisation of 'status_error' for the Win32 error code domain.
10 using win32_error = status_error<win32_code_domain>;
11
12 ///! (Windows only) A NT error code, those returned by NT kernel functions.
13 using nt_code = status_code<nt_code_domain>;
14 ///! (Windows only) A specialisation of 'status_error' for the NT error code domain.
15 using nt_error = status_error<nt_code_domain>;
16
17 /*! (Windows only) A COM error code. Note semantic equivalence testing is only
18 implemented for 'FACILITY_WIN32' and 'FACILITY_NT_BIT'. As you can see at
19 [https://blogs.msdn.microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/](https://blogs.msdn.
    microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/),

```

```

20 there are an awful lot of COM error codes, and keeping mapping tables for all of
21 them would be impractical (for the Win32 and NT facilities, we actually reuse the
22 mapping tables in 'win32_code' and 'nt_code'). You can, of course, inherit your
23 own COM code domain from this one and override the '_equivalent()' function
24 to add semantic equivalence testing for whichever extra COM codes that your
25 application specifically needs.
26 */
27 using com_code = status_code<com_code_domain>;
28 //! (Windows only) A specialisation of 'status_error' for the COM error code domain.
29 using com_error = status_error<com_code_domain>;
30 #endif
31
32 /*! An erased-mutable status code suitably large for all the system codes
33 which can be returned on this system.
34
35 For Windows, these might be:
36
37     - 'com_code' ('HRESULT') [you need to include "com_code.hpp" explicitly for this]
38     - 'nt_code' ('LONG')
39     - 'win32_code' ('DWORD')
40
41 For POSIX, 'posix_code' is possible.
42
43 You are guaranteed that 'system_code' can be transported by the compiler
44 in exactly two CPU registers.
45 */
46 using system_code = status_code<erased<intptr_t>>>;

```

3.8 Proposed `std::error` object

```

1  /*! An erased 'system_code' which is always a failure. The closest equivalent to
2  'std::error_code', except it cannot be null and cannot be modified.
3
4  This refines 'system_code' into an 'error' object meeting the requirements of
5  [P0709 Zero-overhead deterministic exceptions: Throwing values](https://wg21.link/P0709).
6
7  Differences from 'system_code':
8
9  - Always a failure (this is checked at construction, and if not the case,
10 the program is terminated as this is a logic error)
11 - No default construction.
12 - No empty state possible.
13 - Is immutable.
14
15 As with 'system_code', it remains guaranteed to be two CPU registers in size,
16 and trivially copyable.
17 */
18 class error : public system_code
19 {
20     using system_code::clear;
21     using system_code::empty;
22     using system_code::success;
23     using system_code::failure;
24

```

```

25 public:
26     /// The type of the erased error code.
27     using system_code::value_type;
28     /// The type of a reference to a message string.
29     using system_code::string_ref;
30
31     /// Default construction not permitted.
32     error() = delete;
33     /// Copy constructor.
34     error(const error &) = default;
35     /// Move constructor.
36     error(error &&) = default;
37     /// Copy assignment.
38     error &operator=(const error &) = default;
39     /// Move assignment.
40     error &operator=(error &&) = default;
41     ~error() = default;
42
43     /*! Implicit copy construction from any other status code if its value type is
44     trivially copyable and it would fit into our storage.
45
46     The input is checked to ensure it is a failure, if not then
47     'SYSTEM_ERROR2_FATAL()' is called which by default calls 'std::terminate()'.
48     */
49     template <class DomainType>
50     error(const status_code<DomainType> &v) noexcept;
51
52     /*! Implicit construction from any type where an ADL discovered
53     'make_status_code(T &&)' returns a 'status_code' whose value type is
54     trivially copyable and it would fit into our storage.
55
56     The input is checked to ensure it is a failure, if not then
57     'SYSTEM_ERROR2_FATAL()' is called which by default calls 'std::terminate()'.
58     */
59     template <class T>
60     error(T &&v) noexcept(noexcept(make_status_code(std::declval<T>())));
61 };
62
63     /// True if the status code's are semantically equal via 'equivalent()'.
64     template <class DomainType> inline bool operator==(const status_code<DomainType> &a, const error &b)
65     noexcept;
66     /// True if the status code's are not semantically equal via 'equivalent()'.
67     template <class DomainType> inline bool operator!=(const status_code<DomainType> &a, const error &b)
68     noexcept;
69     /// True if the status code's are semantically equal via 'equivalent()' to the generic code.
70     inline bool operator==(const error &a, errc b) noexcept;
71     /// True if the status code's are semantically equal via 'equivalent()' to the generic code.
72     inline bool operator==(errc a, const error &b) noexcept;
73     /// True if the status code's are not semantically equal via 'equivalent()' to the generic code.
74     inline bool operator!=(const error &a, errc b) noexcept;
75     /// True if the status code's are not semantically equal via 'equivalent()' to the generic code.
76     inline bool operator!=(errc a, const error &b) noexcept;

```


4 Design decisions, guidelines and rationale

4.1 Do not `#include <string>`

`<system_error>`, on all the major STL implementations, includes `<string>` as `std::error_code::message()`, amongst other facilities, returns a `std::string`. `std::string`, in turn, drags in the STL allocator machinery and a fair few algorithms and other headers.

Bringing in so much extra stuff is a showstopper for the use of `std::error_code` in the global APIs of very large C++ code bases due to the effects on build and link times. As much as C++ Modules may, or may not, fix this some day, adopting `std::error_code` – which is highly desirable to large C++ code bases which globally disable C++ exceptions such as games – is made impossible. Said users end up having to locally reinvent a near clone of `std::error_code`, but one which doesn't use `std::string`, which is unfortunate.

Moreover, because `<stdexcept>` must include `<system_error>`, and many otherwise very simple STL facilities such as `<array>`, `<complex>`, `<iterator>` or `<optional>` must include `<stdexcept>`, we end up dragging in `<string>` and the STL allocator machinery when including those otherwise simple and lightweight STL headers for no good purpose other than that `std::error_code::message()` returns a `std::string`! That deprives very large C++ code bases of being able to use `std::optional<T>` and other such vocabulary types in their global headers.

Hence, this implicit dependency of `<system_error>` on `<string>` contravenes [P0939]'s admonition *'Note that the cost of compilation is among the loudest reasonable complaints about C++ from its users'*. It also breaks the request *'make C++ easier to use and more effective for large and small embedded systems'* by making a swathe of C++ library headers not [P0829] *Freestanding C++* compatible.

It is trivially easy to fix: stop using `std::string` to return textual representation of codes. This proposed design uses a `string_ref` instead, this is a potentially reference counted handle to a string. It is extremely lightweight, freestanding C++ compatible, and drags in no unnecessary headers.

4.2 All constexpr sourcing, construction and destruction

`<system_error>` was designed before constexpr entered the language, and many operations which ought to be constexpr for such a simple and low-level facility are not. Simple things like the `std::error_code` constructor is not constexpr, bigger things like `std::error_category` are not constexpr, and far more importantly the global source of error code categories is not constexpr, forcing the compiler to emit a magic static initialisation fence, which introduces significant added code bloat as magic fences cannot be elided by the optimiser.

The proposed replacement makes everything which can be constexpr just that. If it cannot be constexpr, it is literal or trivial to the maximum extent possible. Empirical testing has found excellent effects on the density of assembler generated, with recent GCCs and clangs, almost all of the time now the code generated with the replacement design is as optimal as a human assembler writer might write.

4.3 Header only libraries can now safely define custom code categories

Something probably unanticipated at the time of the design of `<system_error>` is that bespoke `std::error_category` implementations are unsafe in header only libraries. This has caused significant, and usually unpleasant, surprise in the C++ user base.

The problem stems from the comparison of `std::error_category` implementations which is *required* by the C++ standard to be a comparison of address of instance. When comparing an error code to an error condition, the `std::error_category::equivalent()` implementation compares the input error code's category against a list of error code categories known to it in order to decide upon equivalence. This is by address of instance.

Header only libraries must use Meyer singletons to implement the source of the custom `std::error_category` implementation i.e.

```
1 inline const my_custom_error_category &custom_category()  
2 {  
3     static my_custom_error_category v;  
4     return v;  
5 }
```

Ordinarily speaking, the linker would choose one of these inline function implementations, and thus `my_custom_error_category` gets exactly one instance, and thus one address in the final executable. All would therefore seem good.

Problems begin when a user uses the header only library inside a shared library. Now there is a single instance of the inline function *per shared library*, not per final executable. It is not uncommon for users to use more than one shared library, and thus multiple instances of the inline function come into existence. You now get the unpleasant situation where there are multiple singletons in the process, each with a different address, despite being the same error code category. Comparisons between error codes and categories thus subtly break in a somewhat chance based, hard to debug, way¹.

Those bitten by this 'feature' tend to be quite bitter about it. This author is one of those embittered. He has met others who have been similarly bitten through the use of ASIO and the Boost C++ Libraries. It's a niche problem, but one which consumes many days of very frustrating debugging for the uninitiated.

The proposed design makes error category sources all-constexpr as well as error code construction. This is incompatible with singletons, so the proposed design does away with the need for singleton sources entirely in favour of stateless code domains with a static random unique 64-bit id, of which there can be arbitrarily many instantiated at once, and thus the proposed design is safe for use in header only libraries.

¹Do inline variables help? Unfortunately not. They suffer from the same problem of instance duplication when used in shared libraries. This is because standard C++ code has no awareness of shared libraries.

In case there is concern of collision in a totally random unique 64 bit id, here are the number of random 64-bit numbers needed in the same process space for various probabilities of collision (note that 10e15 is the number of bits which a hard drive guarantees to return without mistake):

Probability of collision	10e-15	10e-12	10e-9	10e-6	10e-3 (0.1%)	10e-2 (1%)
Random 64-bit numbers needed	190	6100	190,000	6,100,000	190,000,000	610,000,000

4.4 No more `if(!ec)` ...

`std::error_code` provides a boolean test. The correct definition for the meaning of the boolean test is ‘is the value in this error code all bits zero, ignoring the category?’. It does **not** mean ‘is there no error?’.

This may seem like an anodyne distinction, but it causes real confusion. During a discussion on the Boost C++ Libraries list regarding this issue, multiple opinions emerged over whether this was ambiguous, whether it would result in bugs, whether it was serious, whether programmers who wrote the code assuming the latter were the ones at fault, or whether it was the meaning of the boolean test. No resolution was found.

All this suggests to SG14 that there is unhelpful ambiguity which we believe can never lead to better quality software, so we have removed the boolean test in the proposed design. Developers must now be clear as to exactly what they mean: `if(ec.success()) ...`, `if(ec.failure()) ...` and so on.

4.5 No more filtering codes returned by system APIs

Because `std::error_code` treats all bits zero values specially, and its boolean test does not consider category at all, when constructing error codes after a syscall, one must inevitably add some logic which performs a local check of whether the system returned code is a failure or not, and only then follow the error path.

This is fine for a lot of use cases, but many platforms, and indeed third party libraries, like to return success-with-information or success-with-warning codes. The current `<system_error>` does not address the possibility of multiple success codes being possible, nor that there is any success code other than all bits zero.

It also forces the program code which constructs the system code into an error code to be aware of implementation details of the source of the code in order to decide whether it is a failure or not. That is usually the case, but is not always the case. For where it is not the case, forcing this on users breaks clean encapsulation.

The proposed redesign accepts unfiltered and unmodified codes from any source. The category – called a *domain* in this proposal – interprets codes of any form of success or failure. Users can always safely construct a `status_code` (in this proposal, not [P0262](#)’s `status_value`) without knowing anything about the implementation details of its source. No one value is treated specially from any other.

4.6 All comparisons between codes are now semantic, not literal

Even some members of WG21 get the distinction between `std::error_code` and `std::error_condition` incorrect. That is because they appear to be almost the same thing, the same design, same categories, with only a vague documentation that one is to be used for system-specific codes and the other for non-system-specific codes.

This leads to an unnecessarily steep learning curve for the uninitiated, confusion amongst programmers reading code, incorrect choice of `std::error_condition` when `std::error_code` was meant, surprise when comparisons between codes and conditions are semantic not literal, and more of that general ambiguity and confusion we mentioned earlier.

The simple solution is to do away with all literal comparison entirely. Comparisons of `status_code` are **always** semantic. If the user really does want a literal comparison, they can manually compare domain and values by hand. Almost all of the time they actually want semantic comparison, and thus `operator ==`'s non-regular semantic comparison is exactly right.

4.7 `std::error_condition` is removed entirely

As comparisons are now always semantic between `status_code`'s, there is no longer any need for a distinction between `std::error_code` and `std::error_condition`. We therefore simplify the situation by removing any notion of `std::error_condition` altogether.

4.8 `status_code`'s value type is set by its domain

`std::error_code` hard codes its value to an `int`, which is problematic for third party error coding schemes which use a `long`, or even an `unsigned int`. `status_code<DomainType>` sets its `value_type` to be `DomainType::value_type`. Thus if you define your own domain type, its value type can be any type you like, including a structure or class.

This enables *payload* to be transmitted with your status code e.g. if the status code represents a failure in the filesystem, the payload might contain the path of a relevant file. It might contain the stack backtrace of where a failure or warning occurred, a `std::exception_ptr` instance, or anything else you might like.

4.9 `status_code<DomainType>` is type erasable

`status_code<DomainType>` can be type erased into a `status_code<void>` which is an immutable, unrelatable, uncopyable type suitable for passing around by const lvalue reference only. This allows non-templated code to work with arbitrary, unknown, `status_code<DomainType>` instances. One may no longer retrieve their value obviously, but one can still query them for whether they represent success or failure, or for a textual message representing their value, and so on.

If, and only if, `DomainType::value_type` and some type `U` are `TriviallyCopyable` and the size of `DomainType::value_type` is less than or equal to size of `U`, an additional type erasure facility

becomes available, that of `status_code<erased<U>>`. Unlike `status_code<void>`, this type erased form is copyable which is safe as `DomainType::value_type` and `U` are `TriviallyCopyable`, and are therefore both copyable as if via `memcpy()`.

This latter form of type erasure is particularly powerful. It allows one to define some global `status_code<erased<U>>` which is common to all code: `status_code<erased<intptr_t>>` would be a very portable choice². Individual components may work in terms of `status_code<LocalErrorType>`, but all public facing APIs may return only the global `status_code<erased<intptr_t>>`. This facility thus allows any arbitrary `LocalErrorType` to be returned, unmodified, *with value semantics* through code which has no awareness of it. The only conditions are that `LocalErrorType` is trivially copyable, and is not bigger than the erased `intptr_t` type.

4.10 More than one ‘system’ error coding domain: `system_code`

`std::system_category` assumes that there is only one ‘system’ error coding, something mostly true on POSIX, but not elsewhere, especially on Microsoft Windows where at least four primary system error coding schemes exist: (i) POSIX `errno` (ii) Win32 `GetLastError()` (iii) NT kernel `NTSTATUS` (iv) COM/WinRT/DirectX `HRESULT`.

The proposed library makes use of the `status_code<erased<U>>` facility described in the previous section to define a type alias `system_code` to a type erased status code sufficiently large enough to carry any of the system error codings on the current platform. This allows code to use the precise error code domain for the system failure in question, and to return it type erased in a form perfectly usable by external code, which need neither know nor care that the failure stemmed originally from COM, or Win32, or POSIX. All that matters is that the status code semantically compares true to say `std::errc::no_such_file_or_directory`.

4.11 `std::errc` is now represented as type alias `generic_code`

Similar, but orthogonal, to `system_code` is `generic_code` which has a value type of the strongly typed enum `std::errc`. Codes in the generic code domain become the ‘portable error codes’ formerly represented by `std::error_condition` in that they act as semantic comparator of last resort.

Generic codes allow one to write code which semantically compares success or failure to the standard failure reasons defined by POSIX. This allows one to write portable code which works independent of platform and implementation.

5 Technical specifications

No Technical Specifications are involved in this proposal.

²Why? On x64 with SysV calling convention, a trivially copyable object no more than two CPU registers of size will be returned from functions via CPU registers, saving quite a few CPU cycles. AArch64 will return trivially copyable objects of up to 64 bytes via CPU registers!

6 Frequently asked questions

- 6.1 Implied in this design is that code domains must do nothing in their constructor and destructors, as multiple instances are permitted and both must be trivial and constexpr. How then can dynamic per-domain initialisation be performed e.g. setting up at run time a table of localised message strings?**

The simplest is to use statically initialised local variables, though be aware that it is always legal to use status code from within static initialisation and finalisation, so you need to lazily construct any tables on first use and never deallocate. Slightly more complex is to use the domain's `string_ref` instances to keep a reference count of the use of the code domain, when all `string_ref` instances are destroyed, it is safe to deallocate any per-domain data.

7 Acknowledgements

Thanks to Ben Craig, Arthur O'Dwyer and Vicente J. Botet Escriba for their comments and feedback.

8 References

- [N2066] Beman Dawes,
TR2 Diagnostics Enhancements
<https://wg21.link/N2066>
- [P0262] Lawrence Cowl, Chris Mysisen,
A Class for Status and Optional Value
<https://wg21.link/P0262>
- [P0709] Herb Sutter,
Zero-overhead deterministic exceptions: Throwing values
<https://wg21.link/P0709>
- [P0824] O'Dwyer, Bay, Holmes, Wong, Douglas,
Summary of SG14 discussion on <system_error>
<https://wg21.link/P0824>
- [P0829] Ben Craig,
Freestanding proposal
<https://wg21.link/P0829>
- [P0939] B. Dawes, H. Hinnant, B. Stroustrup, D. Vandevorode, M. Wong,
Direction for ISO C++
<http://wg21.link/P0939>

- [P1029] Douglas, Niall
SG14 [[move_relocates]]
<https://wg21.link/P1029>
- [1] *Boost.Outcome*
Douglas, Niall and others
<https://ned14.github.io/outcome/>
- [2] *stl-header-heft github analysis project*
Douglas, Niall
<https://github.com/ned14/stl-header-heft>